



UNIVERSIDADE DE PERNAMBUCO (UPE)

Equipe: Arthur, Guilherme, Pedro, Vinícius Ribeiro e Vinicyus

Relatório - Construção Capibot

Recife

2026

Resumo

O CapiBot é um projeto de robótica móvel voltado à aplicação de tecnologia autônoma em limpeza urbana preventiva. A ideia central é simular e prototipar um robô capaz de percorrer rotas pré-definidas, representar ruas por linhas no ambiente de teste, identificar pontos associados a bueiros e, em uma etapa futura, coletar resíduos pequenos e médios com uma garra robótica.

Ao longo da disciplina, o projeto passou por três níveis de desenvolvimento: concepção do problema urbano, simulação no CoppeliaSim e construção de um protótipo físico. A parte virtual explorou conceitos de Shadow Digital e Gêmeo Digital, enquanto a parte física evoluiu para um robô seguidor de linha com ESP32-CAM, ponte H e motores DC. A versão final documentada não utiliza sensor IR; o seguimento de linha é feito por visão computacional embarcada.

1. Contexto e problema

Recife enfrenta problemas recorrentes ligados ao escoamento urbano, incluindo alagamentos associados ao acúmulo de resíduos e à obstrução de bueiros. Nos slides iniciais do projeto, o problema foi estruturado em três pontos: bueiros entupidos, limpeza manual lenta/perigosa e ausência de manutenção preventiva sistematizada.

O Problema

**Bueiros Entupidos**
Resíduos sólidos bloqueiam o escoamento da água

**Limpeza Ineficiente**
Processo manual: lento, custoso e perigoso

**Sem Prevenção**
Ausência de manutenção preventiva sistematizada



Problema urbano: bueiros, limpeza e prevenção

A proposta do CapiBot surge como uma resposta de prototipagem tecnológica a esse problema: um robô autônomo que percorre rotas definidas, identifica pontos de coleta e, futuramente, realiza remoção automatizada de pequenos resíduos.

2. Objetivos do projeto

2.1 Objetivo geral

Desenvolver, simular e prototipar um robô autônomo de pequeno porte capaz de seguir trajetórias pré-definidas e apoiar a inspeção/coleta de resíduos próximos a bueiros, usando recursos de robótica móvel, visão computacional e simulação.

2.2 Objetivos específicos

- Modelar um ambiente de simulação no CoppeliaSim.
- Implementar um robô seguidor de linha virtual.
- Avaliar a viabilidade de sensores, atuadores e arquitetura de controle.
- Planejar a transição entre simulação e protótipo físico.
- Construir um robô físico com tração diferencial.
- Integrar ESP32-CAM e controle dos motores por ponte H.
- Registrar indicadores de desempenho por semana.
- Identificar limitações e próximos passos para evolução do sistema.

3. Escopo funcional

O escopo foi definido de forma realista para um projeto de disciplina. O CapiBot não foi concebido como veículo urbano de navegação livre, mas como robô de teste em rotas definidas. O material de proposta destaca que o robô segue trajetos pré-definidos, opera em ambiente delimitado, identifica bocas de lobo e detecta resíduos próximos a pontos de coleta.

1. Definição do conceito do CapiBot	
O que o CapiBot FAZ ✓	O que o CapiBot NÃO FAZ ✗
Segue <u>rotas pré-definidas</u> (Visão Computacional / seguidor de linha)	<u>Não navega livremente</u> pela cidade
Opera em <u>ruas asfaltadas</u> com meio-fio definido	Não funciona bem em terrenos irregulares ou sem delimitação
Identifica bocas de lobo ao longo do trajeto	Não reconhece qualquer tipo de infraestrutura urbana mais complexa
Detecta resíduos próximos aos bueiros	Não identifica objetos com alta complexidade ou ambiguidade
Coleta lixo pequeno e médio com garra robótica	<u>Não coleta objetos grandes ou muito pesados</u>
Realiza <u>manutenção preventiva contínua</u>	<u>Não substitui equipes humanas</u>
Funciona de forma autônoma em trajetos definidos	Não toma decisões complexas fora do escopo programado
Pode ser testado e validado em simulação (CoppeliaSim)	Protótipo sem maturação para larga escala

Escopo: o que o CapiBot faz e não faz

3.1 O que o CapiBot faz nesta fase

- Segue rotas pré-definidas por linha.
- Pode ser validado em simulação.
- Usa percepção visual para estimar posição da linha no protótipo físico.

- Possui arquitetura compatível com integração futura de garra e detecção de resíduos.

3.2 O que o CapiBot não faz nesta fase

- Não navega livremente pela cidade.
- Não substitui equipes humanas de limpeza.
- Não reconhece qualquer infraestrutura urbana complexa.
- Não coleta objetos grandes ou pesados.
- Não está maduro para uso em larga escala.

4. Fundamentação técnica: CoppeliaSim, Shadow Digital e Gêmeo Digital

A disciplina trabalhou com a ideia de simulação como etapa anterior à construção física. O CoppeliaSim foi usado como ambiente de simulação para modelagem de robôs, sensores e trajetórias. A API remota ZeroMQ do CoppeliaSim permite controlar uma simulação a partir de aplicações externas ou hardware remoto, o que é coerente com a proposta de integração entre simulação, scripts e protótipo.

O projeto também discutiu a diferença entre Gêmeo Digital e Shadow Digital. Um Gêmeo Digital completo pressupõe sincronização e comunicação bidirecional entre o sistema físico e o virtual. Um Shadow Digital é mais simples: o modelo virtual recebe dados ou serve como representação/validação, mas não necessariamente atua de volta no sistema físico.

Gêmeo Digital vs Shadow Digital

Um **Shadow Digital (Sombra Digital)** é uma representação virtual que recebe dados do sistema físico, mas **sem comunicação de volta** — o fluxo de dados é unidirecional (físico → virtual).

Aspecto	Gêmeo Digital	Shadow Digital
Fluxo de dados	Bidirecional (físico ↔ virtual)	Unidirecional (físico → virtual)
Atuação remota	Sim, o virtual pode controlar o físico	Não — apenas observação e análise
Sincronização	Tempo real contínuo	Tempo real ou periódica
Complexidade	Alta (requer integração total)	Menor (foco em observação)
Custo de implementação	Mais elevado	Mais acessível
Uso típico	Controle operacional, otimização ativa	Monitoramento, análise, simulação prévia

Made with GROBID

Gêmeo Digital vs Shadow Digital

A decisão de trabalhar inicialmente como Shadow Digital foi adequada porque:

- reduz complexidade de integração;
- permite validar algoritmos no CoppeliaSim antes do hardware;
- evita custo e risco de prototipagem prematura;
- mantém caminho aberto para evolução futura para Gêmeo Digital completo.

5. Proposta de virtualização

Na proposta de desenvolvimento, o CoppeliaSim foi apontado como capaz de representar elementos importantes do CapiBot:

- **Vision Sensor:** substitui a ESP32-CAM na simulação.
- **Proximity Sensors:** emulam sensores de linha ou obstáculos.
- **Juntas articuladas:** simulam garra e motores.
- **Ambiente virtual:** representa ruas, bueiros e pontos de coleta.
- **Lua/Python Remote API:** permite controlar a simulação por scripts.



Virtualização no CoppeliaSim

Essa etapa permitiu validar comportamento antes de exigir integração física completa.

6. Cronograma e evolução

O cronograma do projeto foi estruturado em cinco semanas:

Semana	Entrega prevista	Síntese
Semana 1	Simulação	Modelagem do ambiente no CoppeliaSim
Semana 2	Seguidor de linha	Implementação virtual do carro seguidor de linha
Semana 3	Garra	Implementação/modelagem da garra robótica
Semana 4	Visão computacional	Deteção de objetos com ESP32-CAM

Semana	Entrega prevista	Síntese
Semana 5	Integração	Otimização e integração dos sistemas

A fase de simulação entregou ambiente modelado, carro seguidor de linha funcional e sistema de tracking/objetos em ambiente virtual. A fase física concentrou-se na construção do robô, na maquete e no código embarcado da ESP32-CAM.

7. Da simulação ao protótipo físico

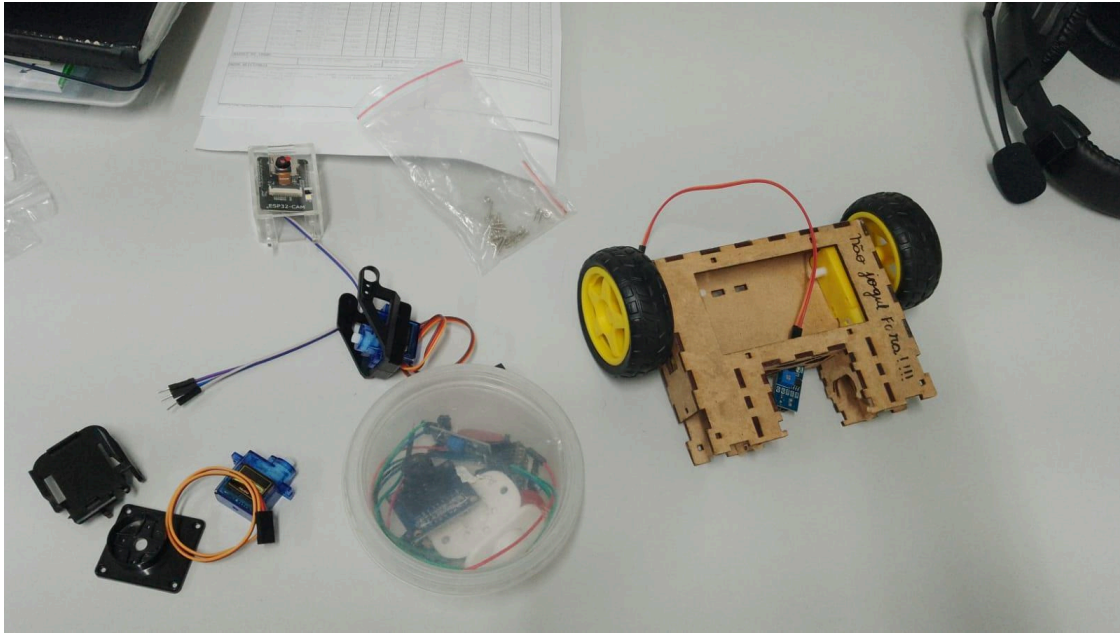
A transição para o físico exigiu uma redução de escopo. Na simulação, foi considerado o uso de três câmeras fixas no ambiente. No protótipo real, a proposta foi simplificada para uma ESP32-CAM embarcada, inicialmente associada à ideia de câmera móvel e sensor IR de linha. Na evolução prática do projeto, o IR foi removido e a câmera passou a assumir também o seguimento de linha.



Da simulação para o físico

7.1 Maquete e chassi

A maquete física foi construída com linhas que representam as ruas percorridas pelo robô. O chassi de MDF com duas rodas e tração diferencial serviu como base para testes de locomoção.



Protótipo físico e componentes

7.2 Componentes físicos

Componente	Papel no protótipo
ESP32-CAM AI Thinker	Captura da imagem e execução do algoritmo embarcado
Ponte H	Controle de sentido e velocidade dos motores
Motores DC com redução	Tração diferencial
Bateria 2S/USB/FTDI	Alimentação e testes
Chassi MDF	Estrutura mecânica
Sensor IR TCRT5000	Testado, mas removido da versão final
Servo/garra	Planejados/modelados, mas não concluídos fisicamente nesta iteração

8. Desenvolvimento eletrônico e decisões de pinagem

A integração elétrica foi uma das partes mais críticas do projeto físico. A ESP32-CAM possui poucos GPIOs disponíveis quando a câmera está em uso, e vários deles têm funções especiais de boot, SD ou PSRAM. Por isso, o mapeamento de motores precisou ser validado por testes específicos.

8.1 Problemas encontrados

- Falha de gravação por GND desconectado entre FTDI e ESP32-CAM.
- Instabilidade associada ao GPIO12, que interferiu no boot/leitura da flash.
- Instabilidade com GPIO16, associado a conflito com câmera/PSRAM.
- Sensor IR ligado em 5 V, gerando preocupação com nível lógico no GPIO da ESP32.

- Necessidade de remover o IR da versão final e reaproveitar GPIO2.

8.2 Pinagem final

```
#define DIR_1 2
#define DIR_2 13

#define ESQ_1 14
#define ESQ_2 15
```

Mapa físico final:

Função	GPIO
Motor direito, entrada 1	GPIO2
Motor direito, entrada 2	GPIO13
Motor esquerdo, entrada 1	GPIO14
Motor esquerdo, entrada 2	GPIO15
Flash da ESP32-CAM	GPIO4, desligado na rotina
IR	Removido
GPIO12	Não usar
GPIO16	Não usar com câmera

Configuração de inversão:

```
bool INVERTER_ESQ = true;
bool INVERTER_DIR = false;
bool INVERTER_CORRECAO = true;
```

9. Software embarcado do robô físico

O código atual está no arquivo:

src/CapiBot_v29_v4_gpio2_inv_final.ino

Ele usa a ESP32-CAM para capturar imagem da pista, processar a região inferior do frame e controlar a ponte H dos motores. A câmera é configurada para uso em tons de cinza, reduzindo custo de processamento. O driver esp32-camera, mantido pela Espressif, fornece a base de suporte a sensores de imagem na família ESP32.

9.1 Fluxo lógico

1. Inicializar Serial, câmera e motores.
2. Capturar frame da ESP32-CAM.
3. Recortar região de interesse da imagem.
4. Calcular limiar de separação claro/escuro.
5. Dividir a ROI em faixas horizontais.
6. Detectar blocos escuros compatíveis com a linha.
7. Rejeitar sombras, bordas e ruídos.
8. Calcular erro lateral e ângulo da linha.
9. Classificar estado do robô.
10. Converter estado/controle em PWM para motores.

9.2 Estados do robô

Estado	Interpretação
LINHA_RETÁ	linha está suficientemente centralizada
CURVA_45_ESQUERDA / CURVA_45_DIREITA	curva moderada
CURVA_90_ESQUERDA / CURVA_90_DIREITA	curva forte ou erro alto
BUSCA_PENDULAR	linha perdida; busca expansiva alternando lados

9.3 Parâmetros centrais

```
int BASE_RETÁ = 100;  
int MAX_RETÁ = 156;  
int MIN_MOTOR = 84;  
  
float KP_ERRO = 0.98;  
float KA_ANGULO = 1.08;  
float KD_ERRO = 0.26;  
float SUAVIZACAO_CONTROLE = 0.40;  
  
int ROI_TOPO_PCT = 38;  
int ROI_BAIXO_PCT = 86;  
int MARGEM_LATERAL_PCT = 7;
```

Esses parâmetros determinam a velocidade base, a força mínima dos motores, o ganho da correção e a região da imagem considerada confiável para detecção da linha.

10. Testes, resultados e indicadores

O documento de KPIs registrou desempenho por semana. As duas primeiras semanas foram consideradas de desempenho total, com modelagem do chassi e seguimento de linha atingindo as metas. A semana 3 teve desempenho parcial: a detecção de objetos escuros atingiu 91%, mas objetos claros ficaram em 48%, prejudicando o indicador geral. A semana 4 teve o principal gap: a modelagem da garra foi concluída, mas sua construção e testes funcionais não foram realizados.

KPI — Indicadores de Desempenho

1. Critérios de Avaliação

- Atingido: resultado igual ou superior à meta definida
- Parcial: resultado entre 50% e 99% da meta
- Não atingido: resultado abaixo de 50% da meta ou atividade não realizada

2. Painel de KPIs por Semana

Semana	Indicador (KPI)	Meta	Resultado	Unidade	Atingiment o	Status
Semana 1	Componentes do chassi modelados	100%	100%	%	100%	Atingido
Semana 1	Tempo de conclusão da modelagem	≤ 5 dias	5 dias	dias	100%	Atingido
Semana 2	Taxa de seguimento da linha	≥ 90%	95%	%	105%	Atingido
Semana 2	Circuitos fechados concluídos	≥ 1	2	circuitos	150%	Atingido
Semana 2	Taxa de erro de trajetória	≤ 10%	5%	%	150%	Atingido
Semana 3	Taxa de detecção geral de objetos	≥ 85%	72%	%	85%	Parcial
Semana 3	Detecção de objetos de cor escura	≥ 90%	91%	%	101%	Atingido
Semana 3	Detecção de objetos de cor clara	≥ 85%	48%	%	56%	Não atingido
Semana 4	Modelagem da garra concluída	100%	100%	%	100%	Atingido
Semana 4	Construção física da garra	100%	0%	%	0%	Não atingido
Semana 4	Funcionalidade da garra testada	≥ 1 teste	0	testes	0%	Não atingido

* Atingimento calculado como: (Resultado ÷ Meta) × 100%

Painel de KPIs

Resumo consolidado:

Semana	Desempenho geral	Observação
Semana 1	100%	modelagem concluída
Semana 2	100%	seguimento de linha em simulação/control e inicial

Semana	Desempenho geral	Observação
Semana 3	33%	dificuldade com detecção de objetos claros
Semana 4	33%	garra física e testes pendentes
Total	64%	7 de 11 KPIs atingidos

3. Objetivo Final do Projeto

O objetivo final do projeto é desenvolver um robô autônomo capaz de realizar a coleta de lixo de forma eficiente, integrando os sistemas de locomoção, detecção e manipulação desenvolvidos ao longo das semanas.

Objetivo Final	Indicador	Meta
Coleta autônoma de lixo	Taxa de coleta sobre objetos detectados	≥ 70%
Trajetórias acessíveis a parques	Conclusão completa de um circuito fechado	Conclusão do circuito

Pré-requisitos para atingir o objetivo final:

- Detecção de objetos com taxa mínima de 70% (incluindo objetos claros) — parcialmente atingido
- Construção e validação funcional da garra mecânica — pendente
- Integração da garra com o sistema de detecção — pendente
- Testes de coleta em circuito real ou simulado — pendente

4. Consolidado por Semana

Semana	KPIs Atingidos	KPIs Parciais	KPIs Não Atingidos	Desempenho Geral
Semana 1	2 / 2	0 / 2	0 / 2	100%
Semana 2	3 / 3	0 / 3	0 / 3	100%
Semana 3	1 / 3	1 / 3	1 / 3	33%
Semana 4	1 / 3	0 / 3	2 / 3	33%
TOTAL	7 / 11	1 / 11	3 / 11	64%

5. Análise dos Resultados

Semanas 1 e 2 — Desempenho total:

- Todos os KPIs foram atingidos nas semanas 1 e 2, demonstrando execução sólida nas etapas de modelagem do chassi e implementação do sensor de linha.
- O robô superou a meta de circuitos fechados (5 concluídos vs. meta de 3), indicando estabilidade no seguimento de trajetória.

Semana 3 — Desempenho parcial:

- A detecção de objetos escuros atingiu a meta (91%). Porém, objetos de cor clara obtiveram apenas 48% de detecção, comprometendo o KPI geral da semana.
- Ponto crítico: ajuste de sensibilidade do sensor para diferentes faixas de luminosidade é necessário.

Semana 4 — Desempenho crítico:

- A modelagem da garra foi concluída (100%), porém a construção física e os testes não foram realizados.
- 2 de 3 KPIs não foram atingidos, o que representa o maior gap de entrega do projeto.

Análise de resultados dos KPIs

11. Análise crítica

11.1 Avanços alcançados

- O problema foi bem delimitado e associado a um contexto urbano relevante.
- O CoppeliaSim permitiu reduzir risco antes da construção física.
- O conceito de Shadow Digital foi aplicado de forma coerente com o estágio do projeto.
- A equipe construiu um protótipo físico e uma maquete de teste.
- O código embarcado alcançou uma versão promissora de seguimento de linha por visão.
- O processo de debug identificou problemas reais de pinagem e alimentação.

11.2 Limitações

- O protótipo físico ainda não possui garra funcional integrada.
- O sistema de coleta de resíduos não foi validado fisicamente.
- A detecção de objetos claros ficou abaixo da meta.
- A ESP32-CAM tem poucos GPIOs livres para uso simultâneo com câmera e ponte H.
- O robô ainda apresenta oscilações na classificação de curvas, principalmente em falsas curvas 90.

11.3 Aprendizados técnicos

O projeto demonstrou que robótica embarcada exige validação incremental. Nem sempre o problema está no algoritmo: houve falhas ligadas a GND comum, pino de boot, conflito de GPIO com câmera/PSRAM e níveis de tensão do sensor IR. A solução mais estável surgiu quando o projeto parou de tentar integrar todos os sensores e focou em uma arquitetura mínima: ESP32-CAM, visão computacional, ponte H, dois motores e pista de linha.

12. Próximos passos

Para uma próxima iteração, recomenda-se:

1. Melhorar a classificação de curvas 90, exigindo confirmação por mais de um frame.
2. Inserir histerese para evitar troca rápida entre esquerda/direita.
3. Remover ou reduzir o uso de ré em curvas fortes.
4. Separar visão e acionamento de motores em duas placas, se a ESP32-CAM continuar limitada por GPIO.
5. Retomar a construção física da garra.
6. Integrar servo/garra apenas após o seguidor de linha ficar estável.
7. Criar testes padronizados para pista reta, curva 45, curva 90 horária e anti-horária.
8. Evoluir de Shadow Digital para Gêmeo Digital parcial, com logs do robô real alimentando a simulação.

13. Conclusão

O CapiBot cumpriu um papel importante como projeto de disciplina: conectou problema urbano, simulação, decisão de escopo, prototipagem física e programação embarcada. O resultado final não é um produto pronto, mas um protótipo documentado com avanços concretos e limitações claras. A etapa de CoppeliaSim validou a viabilidade conceitual; a etapa física expôs desafios reais de eletrônica, pinagem, alimentação e visão computacional.

A principal entrega técnica do robô físico é uma base funcional de seguidor de linha com ESP32-CAM, sem sensor IR, usando controle diferencial dos motores. A principal entrega metodológica é o aprendizado de que simulação, indicadores e testes incrementais são essenciais para transformar uma proposta ambiciosa em um sistema robótico realizável.

Referências e materiais utilizados

- Apresentações do projeto CapiBot enviadas pela equipe: proposta, cronograma, viabilidade, inovação e construção física.
- Documento KPI_Projeto_Robotica.docx com indicadores e análise semanal.
- Código CapiBot_v29_v4_gpio2_inv_final.ino.
- CoppeliaSim User Manual e documentação da ZeroMQ Remote API.
- Espressif esp32-camera driver.
- GitHub Docs — recomendações sobre README e documentação de repositórios.